OFFSIDE LABS

Dynamic Bonding Curve

0.2.2

Smart Contract Security Assessment

October 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie



Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **6**

4 Key Findings and Recommendations **7**

4.1 Incompatible Pool Configurations Can Enter the Transfer Fee Migration Path . 7
4.2 Missing Input Limit Check for Partial Fills 9
4.3 Zero Creator or Fee Claimer Can Block DAMM v2 Migration 12
4.4 Permissionless Leftover Withdrawals Can Lead to Transfer Fee Losses 14
4.5 Transfer Fee Authority Retention After Creator Changes 15
4.6 Minor Rounding Differences in Position Splitting 16
4.7 Quote Fee Changes Can Alter or Block Migration 19
4.8 Informational and Undetermined Issues 20

5 Disclaimer **24**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Dynamic Bonding Curve* smart contracts, starting on September 24th, 2026, and concluding on October 7th, 2026.

Project Overview

Meteora Dynamic Bonding Curve Release 0.2.2 adds Token-2022 transfer fees for base and approved quote tokens across swaps, withdrawals, and DAMM v2 migration. It introduces configurable fee authorities and revises configuration and swap events. Migration adjusts deposits for transfer fees, funds both liquidity positions through one deposit, and assigns unused migration budgets to protocol fees. Transfer fee configurations require constant supply, no token vesting, and customizable compounding migration, with optional fee reinvestment.

Audit Scope

The assessment scope contains mainly the smart contracts of the dynamic-bonding-curve program for the *Dynamic Bonding Curve* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Dynamic Bonding Curve
  - Codebase: `https://github.com/MeteoraAg/dynamic-bonding-curve`
  - Release 0.2.2: `PR-211`
  - Commit Hash: 746806e6a1a1a2306ae0336a8c7659f2b2005e4c

The issues described in this report have been resolved by the following commit:

- Dynamic Bonding Curve
  - Codebase: `https://github.com/MeteoraAg/dynamic-bonding-curve`
  - Commit Hash: 82054dfdffafeb62eac1a158462d0f633bb410e1

We listed the files we have audited below:

- Dynamic Bonding Curve Release 0.2.2
  - programs/dynamic-bonding-curve/src/constants.rs
  - programs/dynamic-bonding-curve/src/error.rs
  - programs/dynamic-bonding-curve/src/event.rs
  - programs/dynamic-bonding-curve/src/instructions/creator/ix_claim_creator_trading_fee.rs
  - programs/dynamic-bonding-curve/src/instructions/creator/ix_withdraw_creator_surplus.rs
  - programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_spl_token.rs

- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_token2022.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_token2022_transfer_hook.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/process_initialize_virtual_pool_with_token2022.rs
- programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/migrate_damm_v2_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/migration/ix_withdraw_migration_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_fee2.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/ix_create_config.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/ix_create_config2.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/ix_create_transfer_hook_config.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/mod.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/process_create_config.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner_trading_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_withdraw_partner_surplus.rs
- programs/dynamic-bonding-curve/src/instructions/swap/ix_swap.rs
- programs/dynamic-bonding-curve/src/instructions/swap/ix_swap2_with_transfer_hook.rs
- programs/dynamic-bonding-curve/src/instructions/swap/mod.rs
- programs/dynamic-bonding-curve/src/instructions/swap/process_swap.rs
- programs/dynamic-bonding-curve/src/instructions/swap/swap_exact_in.rs
- programs/dynamic-bonding-curve/src/instructions/swap/swap_exact_out.rs
- programs/dynamic-bonding-curve/src/instructions/swap/swap_partial_fill.rs
- programs/dynamic-bonding-curve/src/lib.rs
- programs/dynamic-bonding-curve/src/math/safe_math.rs
- programs/dynamic-bonding-curve/src/migration_handler/compounding_liquidity.rs
- programs/dynamic-bonding-curve/src/migration_handler/concentrated_liquidity.rs
- programs/dynamic-bonding-curve/src/migration_handler/mod.rs
- programs/dynamic-bonding-curve/src/state/config.rs

- programs/dynamic-bonding-curve/src/utils/accounts.rs
- programs/dynamic-bonding-curve/src/utils/mod.rs
- programs/dynamic-bonding-curve/src/utils/token.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 5 medium issues
- 2 low issues
- 5 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Incompatible Pool Configurations Can Enter the Transfer Fee Migration Path	Medium	Fixed
02	Missing Input Limit Check for Partial Fills	Medium	Fixed
03	Zero Creator or Fee Claimer Can Block DAMM v2 Migration	Medium	Fixed
04	Permissionless Leftover Withdrawals Can Lead to Transfer Fee Losses	Medium	Fixed
05	Transfer Fee Authority Retention After Creator Changes	Medium	Acknowledged
06	Minor Rounding Differences in Position Splitting	Low	Fixed
07	Quote Fee Changes Can Alter or Block Migration	Low	Acknowledged
08	Price Differences From Deposit Rounding	Informational	Fixed
09	Inaccurate Lamport Refund Comment	Informational	Fixed
10	Transfer Fees Are Classified as Nonzero Despite a Zero Fee Cap	Informational	Fixed
11	Configs With Default Partner Can Fail Token-2022 Pool Initialization	Informational	Fixed
12	New Transfer Hook Configs Reject Quote Mints With Fee Authorities	Informational	Fixed

4 Key Findings and Recommendations

4.1 Incompatible Pool Configurations Can Enter the Transfer Fee Migration Path

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The `migrate_damm_v2` IX selects `process_migrate_damm_v2_with_transfer_fee` when the base mint charges a transfer fee or the quote mint satisfies `has_transfer_fee_or_config_authority` . For the quote mint, retaining a `transfer_fee_config_authority` is sufficient, even when its current and pending transfer fees are zero.

```
651     if has_base_transfer_fee
652         ||
    ↪ has_transfer_fee_or_config_authority(&ctx.accounts.quote_mint.to_account_info())?
653     {
654         process_migrate_damm_v2_with_transfer_fee(ctx, base_transfer_fee,
    ↪ quote_transfer_fee)
655     } else {
656         process_migrate_damm_v2(ctx)
657     }
```

programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_-v2/migrate_damm_v2_initialize_pool.rs#L651-L657

The `process_migrate_damm_v2_with_transfer_fee` handler assumes constant token supply, no base token vesting, and `Customizable` migration with Compounding liquidity. It always uses `CompoundingLiquidity` for its calculations and omits the base-token burn performed by the original handler. These assumptions do not hold for every config that can reach this path:

- **Legacy configs**: Previous release allowed a quote mint with a valid token badge to retain its `transfer_fee_config_authority` while charging zero transfer fees. Such configs could specify different `pre_migration_token_supply` and `post_migration_token_supply` , include base token vesting, or use a non-Compounding liquidity mode. After the upgrade, both existing pools and new pools initialized from these configs can enter the transfer fee handler without any fee settings changing.
- **Quote mint closure and recreation**: Configs created under current release can also be affected. A quote mint with an active `MintCloseAuthority` can pass validation through a token badge. If it initially has no transfer fees or transfer fee configuration authority, a config without base transfer fees can be created without the additional restrictions. While the quote mint's supply is zero, a party controlling its close authority and original address

can close it and recreate it with transfer fees or a fee configuration authority. The existing badge still matches the mint address, and pool initialization does not recheck whether the stored config meets the transfer fee restrictions.

The missing burn also affects the unchanged `withdraw_leftover` IX:

- For **fixed-supply** configs where `pre_migration_token_supply > post_migration_token_supply`, the original handler burns the remaining base tokens up to the configured supply reduction. If migration through the new handler succeeds, tokens intended for burning can instead be included in the balance transferred to `leftover_receiver`.
- For **non-fixed-supply** configs, the original handler keeps the unclaimed fees in the vault and burns the remaining base tokens. The new handler can leave a balance outside the recorded fee amounts in the base vault. Since `withdraw_leftover` only accepts fixed-supply configs, this remainder cannot be withdrawn through that IX

Impact

Affected **fixed-supply** pools may allow `leftover_receiver` to withdraw tokens intended for burning, leaving the post-migration supply higher than intended.

Non-fixed-supply pools may retain unburned tokens in the DBC vault without an existing withdrawal IX for that remainder.

Applying Compounding calculations while DAMM v2 initializes a different liquidity model can also prevent migration or produce liquidity and position allocations inconsistent with the configured model.

Recommendation

It is recommended to:

- Use `get_migration_handler` in both migration paths to select calculations from the configured fee mode. The two position creation flows can stay separate.
- Restore the original burn and leftover handling in `process_migrate_damm_v2_with_transfer_fee`, ensuring tokens intended for burning are not assigned to additional protocol migration fees.
- Require the quote mint's `MintCloseAuthority` to be absent or revoked during config creation and pool initialization, even when a valid token badge is supplied.
- Recheck the quote mint in all pool initialization IXs. If `has_transfer_fee_or_config_authority` returns true, require the stored config to meet the transfer fee restrictions enforced by `create_config2`: constant token supply, no base token vesting, Customizable migration, and Compounding liquidity.

Mitigation Review Log

Fixed in the commit 82054dfdffafeb62eac1a158462d0f633bb410e1.

Meteora Team: We have documented the risk associated with MintCloseAuthority. In general, we recommend avoiding token badges for mints with an active MintCloseAuthority.

Offside Labs: The main issue has been fixed. The team has accepted and documented the remaining MintCloseAuthority risk.

4.2 Missing Input Limit Check for Partial Fills

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

In `process_swap_partial_fill`, the caller supplies `amount_in` as the maximum input available for the swap. For a quote-to-base swap with an `active RateLimiter`, the program first deducts trading fees from this input. If the curve consumes only part of the remaining net amount, `get_swap_result_from_partial_input` recalculates the fee numerator from the amount actually consumed and reconstructs the corresponding fee- inclusive input.

Due to integer rounding in the RateLimiter inverse calculation, the reconstructed fee-inclusive input can exceed the original input even though the curve consumes less net input. The inverse calculation does not itself enforce the caller's input budget. RateLimiter inverse calculation

The PartialFill handler includes any applicable token transfer fee and passes the resulting included_transfer_fee_amount_in to the transfer logic without checking it against the caller's original amount_in. Consequently, the output slippage check can pass while the actual input debit exceeds the supplied budget. This issue can occur even without a token transfer fee. Input reconstruction, token transfer

Current pool initialization rejects RateLimiter configs, limiting the affected path to existing pools whose RateLimiter remains active.

Impact

Affected PartialFill swaps can transfer more quote tokens from the user's account than the specified amount_in. If the account balance is insufficient, the transaction fails.

Proof of Concept

Consider a quote-to-base partial fill on an existing pool while its `RateLimiter is active`. Assume zero dynamic fee and no token transfer fee. All amounts are in the quote token's smallest units, and `ceil` means rounding up to an iteger.

Parameter	Value
User's `amount_in`	2,000,000,000

Parameter	Value
RateLimiter `reference_amount`	1,000,000,000
Initial fee rate	50%
Fee increment per tier	25%
Fee denominator	1,000,000,000

1. Calculate the available net input.

The first 1,000,000,000 units incur a 50% fee, and the next 1,000,000,000 incur a 75% fee:

```
Trading fee = 500,000,000 + 750,000,000
            = 1,250,000,000

Available net input = 2,000,000,000 - 1,250,000,000
                    = 750,000,000
```

Assume the curve consumes **749,999,999**, leaving **1** unit of net input unused. The PartialFill must now reconstruct the total input needed to cover this consumed amount and its trading fee.

2. Reconstruct the fee-inclusive input inside the RateLimiter.

The inverse calculation first estimates a fee-inclusive input of **1,999,999,992**. Applying the forward fee calculation to this estimate gives:

Calculation	Result
Tier-based fee: 500,000,000 + 999,999,992 × 75%	1,249,999,994
Fee numerator: ceil(1,249,999,994 × 1,000,000,000 / 1,999,999,992)	625,000,000
Fee after applying this numerator	1,249,999,995
Net input: 1,999,999,992 − 1,249,999,995	749,999,997

The required net input is 749,999,999, so the estimate is short by **2**. At the current marginal fee rate of 75%, covering these 2 units requires another **8** units:

```
Additional input = ceil(2 / (1 - 75%)) = 8

Reconstructed input = 1,999,999,992 + 8
                    = 2,000,000,000
```

At this point, the reconstructed input still respects the user's budget.

3. Convert the result back into a fee numerator.

The RateLimiter returns a fee numerator rather than the reconstructed input amount:

```
Implied fee = 2,000,000,000 - 749,999,999
            = 1,250,000,001

Fee numerator = ceil(1,250,000,001 × 1,000,000,000 / 2,000,000,000)
              = 625,000,001
```

This represents a fee rate of **62.5000001%**, slightly above the exact implied rate of **62.50000005%**.

4. Reconstruct the input again using that numerator.

The partial-fill calculation uses the returned numerator to calculate the final fee-inclusive input:

```
Final input = ceil(749,999,999 × 1,000,000,000
                  / (1,000,000,000 - 625,000,001))
            = 2,000,000,003
```

With no token transfer fee, this is also the amount passed to the token transfer.

Result	Amount
User's input budget	2,000,000,000
Final transfer amount	2,000,000,003
Amount exceeding the budget	**3**

The swap can therefore transfer **3 units more than the user specified**, provided the account has sufficient funds and the other swap checks pass.

Recommendation

It is recommended to require `included_transfer_fee_amount_in <= amount_in` in `process_swap_partial_fill`, using the caller's original `amount_in`, and return an error if this check fails. This check should occur before applying the swap result or transferring tokens.

- PartialFill swaps whose reconstructed input exceeds the budget will then revert, including cases where the excess is only a few token units.
- Other partial fills can continue normally.

Mitigation Review Log

Fixed in the commit 53dbdb78e6121f9689efda0a883decc45e95bd1e.

4.3 Zero Creator or Fee Claimer Can Block DAMM v2 Migration

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: DoS

Description

For pools whose base mint charges a transfer fee, `process_migrate_damm_v2_with_transfer_fee` can transfer the mint's transfer_fee_config_authority to the current pool creator or the config's fee_claimer, depending on the configured authority option.

`get_migrated_authority` wraps the selected address in Some without checking whether it is the zero address:

```
473     pub fn get_migrated_authority(&self, creator: Pubkey, partner: Pubkey)
            -> Option<Pubkey> {
474         match *self {
475             MigratedTransferFeeAuthorityOption::Immutable
476             | MigratedTransferFeeAuthorityOption::RevokeZeroFee => None,
477             MigratedTransferFeeAuthorityOption::Creator => Some(creator),
478             MigratedTransferFeeAuthorityOption::Partner => Some(partner),
479         }
480     }
```

programs/dynamic-bonding-curve/src/state/config.rs#L473-L480

However, Token-2022 stores this authority as `MaybeNull<Address>`. When processing `SetAuthority`, it rejects `Some(Pubkey::default())` with `InvalidArgument`. This differs from None, which successfully revokes the authority.

```
839                 AuthorityType::TransferFeeConfig => {
840                     let extension =
                            mint.get_extension_mut::<TransferFeeConfig>()?;
841                     let maybe_transfer_fee_config_authority:
                            Option<Address> =
842                         extension.transfer_fee_config_authority.into();
843                     let transfer_fee_config_authority =
                            maybe_transfer_fee_config_authority
844                         .ok_or(TokenError::AuthorityTypeNotSupported)?;
845                     Self::validate_owner(
846                         program_id,
847                         &transfer_fee_config_authority,
848                         authority_info,
849                         authority_info_data_len,
850                         account_info_iter.as_slice(),
851                     )?;
852                     extension.transfer_fee_config_authority =
    new_authority.try_into()?;
```

program/src/processor.rs#L839-L852

```rust
impl TryFrom<PodCOption<Address>> for MaybeNull<Address> {
    type Error = ProgramError;
    fn try_from(p: PodCOption<Address>) -> Result<Self, Self::Error> {
        match p {
            PodCOption {
                option: PodCOption::<Address>::SOME,
                value,
            } if value == Address::default() =>
              ↪ Err(ProgramError::InvalidArgument),
```

interface/src/pod.rs#L242-L249

The selected address can become zero through either of the following paths:

- **Creator**: Pool initialization requires a creator signer, but `transfer_pool_creator` subsequently allows the current creator to assign the zero address (`Pubkey::default()`) during the bonding curve phase. The IX only requires the new address to differ from the current creator.
- **Partner**: Config creation accepts `fee_claimer` without requiring a signature or checking for the zero address (`Pubkey::default()`). A pool can still initialize successfully when the other mint authorities use valid addresses.

In either case, selecting the corresponding migration authority option causes the authority transfer to fail. The error propagates through the handler and rolls back the entire migration transaction, including DAMM v2 pool initialization.

Impact

Once the bonding curve completes, affected pools cannot migrate to DAMM v2 or resume DBC swaps, leaving the funds intended for migration in the DBC vaults.

Existing IXs provide no recovery path: a zero creator cannot sign another creator transfer, and the config's fee_claimer cannot be updated. Retrying migration therefore continues to fail unless the program is changed.

Recommendation

It is recommended to return `None` from `get_migrated_authority` when the selected creator or partner is the zero address (`Pubkey::default()`). This allows migration to complete by permanently revoking the transfer_fee_config_authority while preserving the existing transfer fee basis points and maximum fee.

Mitigation Review Log

Fixed in the commit 82054dfdffafeb62eac1a158462d0f633bb410e1.

4.4 Permissionless Leftover Withdrawals Can Lead to Transfer Fee Losses

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The `withdraw_leftover` IX is permissionless and allows anyone to transfer all remaining base tokens, after excluding unclaimed fees, to the configured `leftover_receiver` ATA once migration to DAMM v2 completes. It does not require the receiver's signature or enforce a minimum amount received after transfer fees. After the transfer succeeds, the pool marks the leftover as withdrawn.

With transfer fees enabled on the base mint, the amount received depends on when this permissionless withdrawal executes:

- When `RevokeZeroFee` is selected, migration schedules a zero transfer fee and revokes the `transfer_fee_config_authority` . Token-2022 activates the zero fee at the migration epoch plus two. The previous fee remains effective until then. Anyone can trigger `withdraw_leftover` during this interval, forcing the receiver to pay a fee that could have been avoided by waiting until the zero fee becomes effective.
- When the `transfer_fee_config_authority` is transferred to the creator or partner, that authority can increase the base transfer fee after migration. If the leftover remains unclaimed when the increased fee becomes effective, anyone can trigger its withdrawal under the higher fee.

The delayed fee update is enforced by Token-2022. Its transfer implementation allows the amount received after fees to be zero and records the fee in the destination account's withheld balance. That balance is controlled by the mint's withheld fee withdrawal authority, independently of the destination account owner. If the same creator or partner controls both fee authorities and the leftover belongs to another party, they can raise the fee, trigger the withdrawal, and collect the withheld tokens. Transfer processing, withheld fee withdrawal.

Impact

A leftover receiver can be forced to pay avoidable transfer fees without approving the withdrawal.

If the same party controls both the base mint's transfer fee configuration authority and its withheld fee withdrawal authority, they can set a 100% transfer fee with a sufficiently large maximum fee. Once the fee becomes effective, they can trigger `withdraw_leftover` and collect the entire amount as withheld fees. The receiver receives no spendable tokens, but DBC marks the leftover as withdrawn.

Recommendation

It is recommended to allow permissionless calls to `withdraw_leftover` only when the transfer fee on the withdrawal is zero. Otherwise, require authorization from `leftover_receiver` .

For `RevokeZeroFee` , permissionless withdrawal should wait until the zero fee becomes effective.

Mitigation Review Log

Fixed in the commit bf7d1cc331601b4e03322f5d07f67a3cc0a3dc4a.

4.5 Transfer Fee Authority Retention After Creator Changes

Severity: Medium	Status: Acknowledged
Target: Smart Contract	Category: Logic Error

Description

`transfer_pool_creator` IX allows the creator role to be transferred before graduation or after migration to DAMM v2. However, it only updates `pool.creator` and does not update the token authorities previously assigned to that creator.

The following authorities can remain with the previous creator:

- Token-2022 TransferFee.withdraw_withheld_authority
  - `TransferFeeWithheldAuthority::Creator` assigns this authority during mint initialization.
- Token-2022 TransferFee.transfer_fee_config_authority
  - `MigratedTransferFeeAuthorityOption::Creator` assigns this authority at graduation. A subsequent creator transfer leaves it unchanged.
- Token-2022 TokenMetadata.update_authority
  - `CreatorUpdateAuthority` or `CreatorUpdateAndMintAuthority` assigns this authority during initialization.
- Token-2022 TokenMetadata.update_authority
  - The same creator metadata options assign this authority during initialization.
- Metaplex metadata update authority for SPL tokens
  - The token authority option assigns metadata control to the creator.
- Base token mint MintTokens authority
  - Transfer Hook pools using `CreatorUpdateAndMintAuthority` assign minting authority to the creator.

The transfer fee authorities are assigned during mint initialization and DAMM v2 migration. The remaining authorities are assigned by the Token-2022 initialization flow and Meta-

plex metadata initialization.

In each case, transferring the DBC creator role leaves the separately stored authority unchanged unless it is transferred or revoked independently.

Impact

Depending on the configured authorities,

- the previous creator can retain permission to withdraw withheld transfer fees, change transfer fee parameters, modify token metadata, redirect the metadata pointer, or mint additional base tokens.
- the new creator receives the DBC role without receiving these token management permissions.

Recommendation

It is recommended to transfer the following creator-assigned authorities to the new creator when updating `pool.creator`:

- The Token-2022 withheld-fee withdrawal authority.
- The Token-2022 transfer fee configuration authority, when it has been assigned to the creator at migration.
- The Token-2022 metadata update authority.
- The Token-2022 metadata pointer authority.
- The Metaplex metadata update authority for SPL tokens.
- The base token mint authority for Transfer Hook pools using `CreatorUpdateAndMintAuthority`.

Mitigation Review Log

Meteora Team: This is intended design. Authorities held by the creator's wallet (such as Token-2022 and Metaplex metadata authorities) are not moved. They can be transferred separately with the instructions from the respective programs.

4.6 Minor Rounding Differences in Position Splitting

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Precision

Description

In `process_migrate_damm_v2_with_transfer_fee`, the program deposits the migration funds into one DAMM v2 position, then splits its distributable liquidity between the partner

and creator. The second position's planned allocation is converted into a split numerator with a denominator of 1_000_000_000, and rounding down:

```
1019    let liquidity_numerator_for_second_position: u32 =
        ↪ safe_mul_div_cast_u128(
1020        second_position_liquidity_distribution.get_total_liquidity()?,
1021        SPLIT_POSITION_DENOMINATOR.into(),
1022        distributable_liquidity,
1023        Rounding::Down,
1024    )?
1025    .safe_cast()?;
```

programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_-v2/migrate_damm_v2_initialize_pool.rs#L1019-L1025

DAMM v2 rounds down again when applying this numerator.

```
365    pub fn get_unlocked_liquidity_by_numerator(&self, numerator: u32) ->
       ↪ Result<u128> {
366        let liquidity_delta = safe_mul_div_cast_u128(
367            self.unlocked_liquidity,
368            numerator.into(),
369            SPLIT_POSITION_DENOMINATOR.into(),
370            Rounding::Down,
371        )?;
372
373        Ok(liquidity_delta)
374    }
```

programs/cp-amm/src/state/position.rs#L365-L374

The second position can therefore receive less liquidity than its planned allocation, with the difference remaining in the first position.

After splitting, DBC adjusts the second position's distribution to its actual balance. However, `adjust_liquidity` preserves its planned unlocked amount where possible and uses the remaining liquidity for permanent locks and vesting. A shortfall can therefore reduce the amount locked.

Meanwhile, the first position is locked using its original distribution, leaving the additional liquidity unlocked. The split can consequently both shift liquidity between the two owners and reduce their combined locked liquidity.

Impact

The second position can receive a slightly smaller share, while the first retains additional unlocked liquidity.

Proof of Concept

Assume the distributable liquidity is 101, with these configured percentages:

- Creator: 50% unlocked.
- Partner: 25% unlocked and 25% permanently locked.

`get_liquidity_distribution` allocates `25 unlocked + 25 locked = 50` to the partner. The creator receives the remaining `51 unlocked` and becomes the first position.

The second position's split numerator is:

```
floor(50 × 1,000,000,000 / 101) = 495,049,504
```

DAMM v2 then calculates the liquidity to transfer:

```
floor(101 × 495,049,504 / 1,000,000,000) = 49
```

The partner receives 49 instead of 50, leaving 52 instead of 51 in the creator's position.

`adjust_liquidity` preserves the partner's 25 unlocked units, leaving only 24 for permanent locking. The creator's extra unit remains unlocked.

Owner	Planned unlocked	Resulting unlocked	Planned permanently locked	Resulting permanently locked
Creator - 1st position	51	52	0	0
Partner - 2nd position	25	25	25	24
Total	**76**	**77**	**25**	**24**

Total liquidity remains 101. One unit moves from the partner's planned permanently locked allocation to the creator's unlocked allocation.

Recommendation

It is recommended to:

- Calculate the split numerator directly by adding the second owner's configured unlocked, permanently locked, and vesting percentages, then multiplying by `SPLIT_POSITION_DENOMINATOR / 100`.
- Update both distributions using the actual position balances after splitting, and handle any remaining rounding differences according to the intended allocation and locking rules.

Mitigation Review Log

Fixed in the commit d4ce3da5526ad3e33107309d06419777fe9ee745.

4.7 Quote Fee Changes Can Alter or Block Migration

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Token

Description

During config creation, `process_create_config` calculates migration deposits using the quote mint's currently effective transfer fee. It verifies that the resulting liquidity exceeds the required dead_liquidity and that the calculated sqrt_root price remains within 1% of the bonding curve's migration sqrt_root price.

```
39          // Verify compounding-derived sqrt price is within 1% of
             ↪  curve-derived sqrt price:
40          // abs_diff / max <= 1 / 100
41          // abs_diff       <= max / 100
42          // abs_diff * 100 <= max
43          require!(
44              migration_sqrt_price.abs_diff(sqrt_price).safe_mul(100)?
45                  <= migration_sqrt_price.max(sqrt_price),
46              PoolError::InvalidCurve
47          );
```

programs/dynamic-bonding-curve/src/migration_handler/compounding_-liquidity.rs#L39-L47

Quote mints with transfer fees or an active `transfer_fee_config_authority` require an operator-created token badge. This approval relies on trust in the quote mint's management. It does not restrict subsequent fee changes. The effective fee may change before migration through an authority update or a previously scheduled change becoming active.

Migration reads the current transfer fee and recalculates the deposits. To preserve the intended token ratio, it reduces the deposit on the side less affected by transfer fees to match the reduction on the other side. Any migration budget left unused after the actual transfers is added to the protocol's migration fees.

Migration does not repeat the relative price check performed during config creation. Approval of the quote mint and successful config creation therefore do not guarantee the originally calculated deposit amounts, price tolerance, or ability to graduate.

Impact

Depending on the effective fee and available amounts, the following outcomes are possible:

1. **Reduced liquidity allocated to the partner and creator**
   If an increased quote fee becomes the limiting factor, fewer quote tokens reach DAMM v2, and the base deposit is reduced accordingly. Both positions receive less liquidity than

anticipated during config creation, even though their allocation percentages remain unchanged.

2. **A substantial part of the migration budget allocated to protocol fees**
   Suppose the budgets after regular migration fees are 10,000 base and 10,000 quote. If base has no transfer fee and quote charges 90%, without the maximum fee limiting that charge, DAMM v2 receives 1,000 of each token. The remaining 9,000 base is recorded as additional protocol migration fees. The 9,000 quote withheld as Token-2022 transfer fees is separate from this protocol allocation. Thus, a successful migration can leave little liquidity in DAMM v2 while assigning most of the base migration budget to the protocol.
3. **Migration at a price outside the original tolerance**
   Rebalancing preserves the budget ratio before integer rounding. When increased fees leave small deposit amounts, rounding can materially change that ratio. The resulting price can exceed the original sqrt_price tolerance while still satisfying the requirements needed to initialize the pool.
4. **Failure to graduate after the bonding curve completes**
   A sufficiently high fee can leave zero net quote tokens, cause the paired base amount to round to zero, or reduce liquidity below the required minimum. For example, a 100% quote fee with a maximum fee covering the entire quote budget leaves no quote tokens for migration. Once the bonding curve is complete, DBC swaps are also rejected, leaving the pool unable to continue curve trading or initialize its DAMM v2 pool under those fee settings.

Recommendation

It is recommended to document the reliance on the approved quote mint's fee management, including possible migration failures, reduced liquidity, price deviations, and substantial additional protocol fees from unused migration budgets.

Mitigation Review Log

Meteora Team: Acknowledged, add comments and docs to describe the limitation here.

4.8 Informational and Undetermined Issues

Price Differences From Deposit Rounding

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Math

In `CompoundingLiquidity::get_migration_deposit_amounts`, the migration amounts are adjusted after transfer fees to approximate the original ratio, `quote_budget / base_budget`. Both branches use `Rounding::Down`, but they affect this ratio in opposite directions.

- When **base** retains the smaller fraction of its budget, the function rounds the adjusted quote amount down, producing a quote-to-base ratio at or below the budget ratio.
- When **quote** retains the smaller fraction, it rounds the adjusted base amount down, producing a ratio at or above the budget ratio.

These calculations keep the returned amounts within the available balances, but the price implied by those amounts can move in either direction relative to the budget ratio. Each division discards less than one smallest token unit, whose relative effect depends on the amount being rounded. Integer rounding can cause the returned deposit ratio to be lower or higher than the original budget ratio, depending on which amount is rounded down.

It is recommended to clarify that the calculation approximately preserves the budget ratio, with rounding potentially shifting the implied price in either direction.

Mitigation Review Log

Fixed in the commit 8e8473ffefb96c61aa97d8d2f95434682c580ad4.

Inaccurate Lamport Refund Comment

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

The comment in `create_account` states that existing lamports are returned to the payer before creating the account.

```
26      // If the account being initialized already has lamports, then
27      // return them all back to the payer so that the account has
28      // zero lamports when the system program's create instruction
29      // is eventually called.
```

programs/dynamic-bonding-curve/src/utils/accounts.rs#L26-L29

The implementation instead retains the existing balance, transfers any additional lamports needed for rent exemption, and then allocates space and assigns ownership. No refund occurs.

It is recommended to update the comment to describe the actual handling of prefunded accounts.

Mitigation Review Log

Fixed in the commit 81e7888ed57cedb6846b671d0751c1c094d454e2.

Transfer Fees Are Classified as Nonzero Despite a Zero Fee Cap

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

`is_transfer_fee_zero` checks the fee bps but ignores `maximum_fee` .

When `maximum_fee` is zero, Token-2022 charges no transfer fee, even if the rate is nonzero. DBC can therefore require a token badge and apply extra config restrictions to a quote mint that charges no transfer fee and has no authority to enable one.

It is recommended to consider a fee setting zero when either its `transfer_fee_basis_points` or `maximum_fee` is zero. Check current and pending settings separately, and keep the existing check for the authority to change fees.

Mitigation Review Log

Fixed in the commit 64300cafc8ea4f36cc5e9cde4954e9e86fb9ad9c.

Configs With Default Partner Can Fail Token-2022 Pool Initialization

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

Config creation accepts a zero `fee_claimer` even when it is selected to receive mint authorities that cannot be assigned to the zero address(`Pubkey::default()`). As a result, a config can be created successfully while subsequent Token-2022 pool initialization fails in the following cases:

- **TransferFee.withdraw_withheld_authority**: When the base mint has a transfer fee and `TransferFeeWithheldAuthority::Partner` is selected, initialization passes `Some(Pubkey::default())` as the withdrawal authority. Token-2022 `rejects` this value when initializing the transfer fee config.
- **Metadata pointer authority**: When `PartnerUpdateAuthority` or the transfer-hook-specific `PartnerUpdateAndMintAuthority` is selected, `get_update_authority` returns `Some(Pubkey::default())` . Token-2022 `rejects` the subsequent `SetAuthority` cpi call.
- **Metadata update authority**: The same selected address is converted locally using `OptionalNonZeroPubkey::try_from` , which also rejects `Some(Pubkey::default())` . In the current execution order, the metadata pointer authority update fails first, so this conversion is not reached.

The initialization TX reverts without leaving an active pool for an existing pool.

It is recommended to validate these authority selections during config creation and reject a `Pubkey::default()` fee_claimer when it is assigned one of the affected authorities, using a clear configuration error.

Mitigation Review Log

Fixed in the commit 82054dfdffafeb62eac1a158462d0f633bb410e1.

New Transfer Hook Configs Reject Quote Mints With Fee Authorities

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

In release 0.2.2, `create_config_with_transfer_hook` rejects quote mints that retain a `transfer_fee_config_authority`, even when their current and scheduled transfer fees are zero and a valid token badge is supplied. Release 0.2.1 accepted this combination. Although `create_config2` accepts such quote mints, it does not support configuring a transfer hook for the base token. Integrations therefore cannot create equivalent new configs using either entry point.

This restriction applies to new config creation. The pool initialization entry point does not apply the same authority check to existing TransferHook configs.

It is recommended to explicitly document this compatibility restriction and clarify that `create_config2` is not a replacement for `create_config_with_transfer_hook`.

Mitigation Review Log

Fixed in the commit 82054dfdffafeb62eac1a158462d0f633bb410e1.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

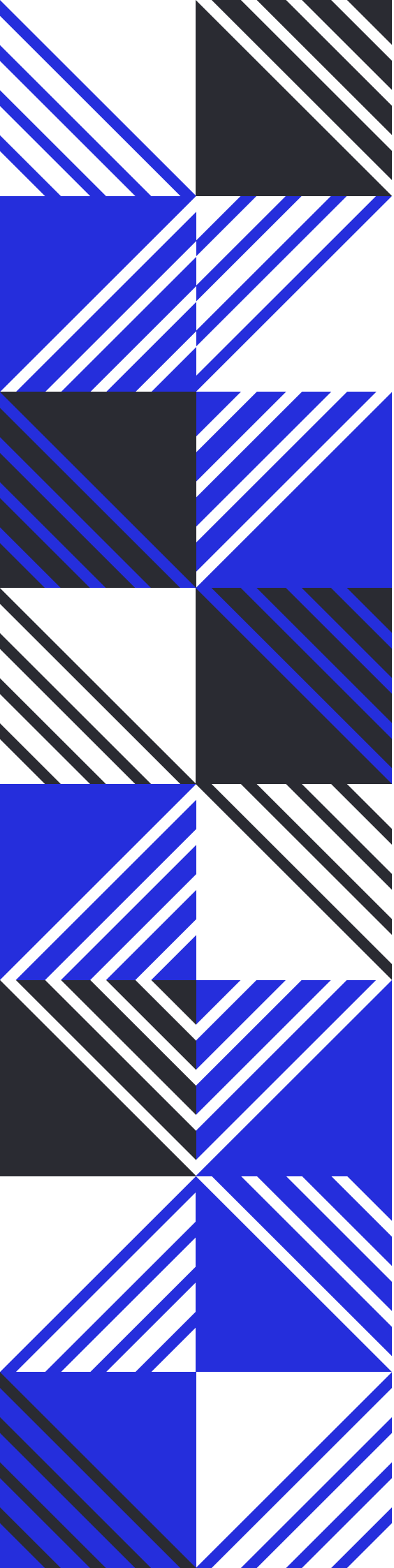
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs